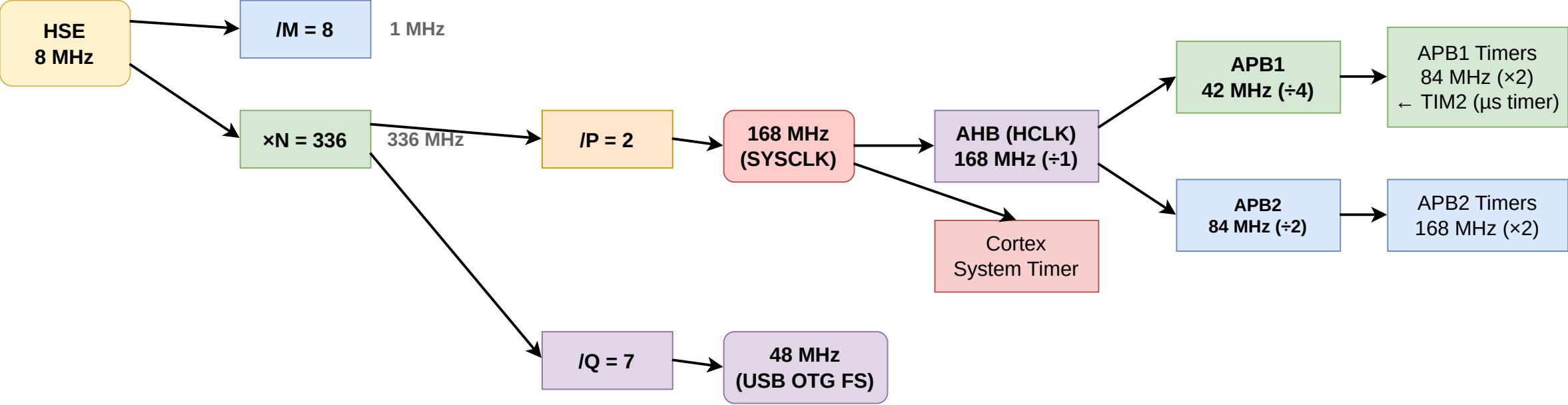


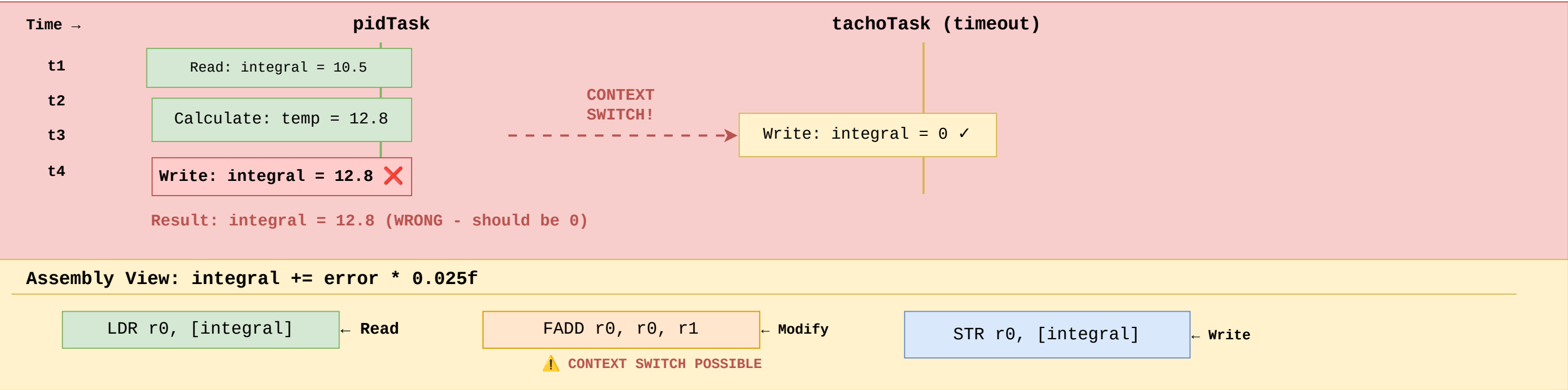
System Clock Configuration



PLL Configuration:
HSE = 8 MHz
M = 8, N = 336, P = 2, Q = 7
VCO = 336 MHz
SYSCLK = 168 MHz
USB = 48 MHz

Race Condition & Thread Safety

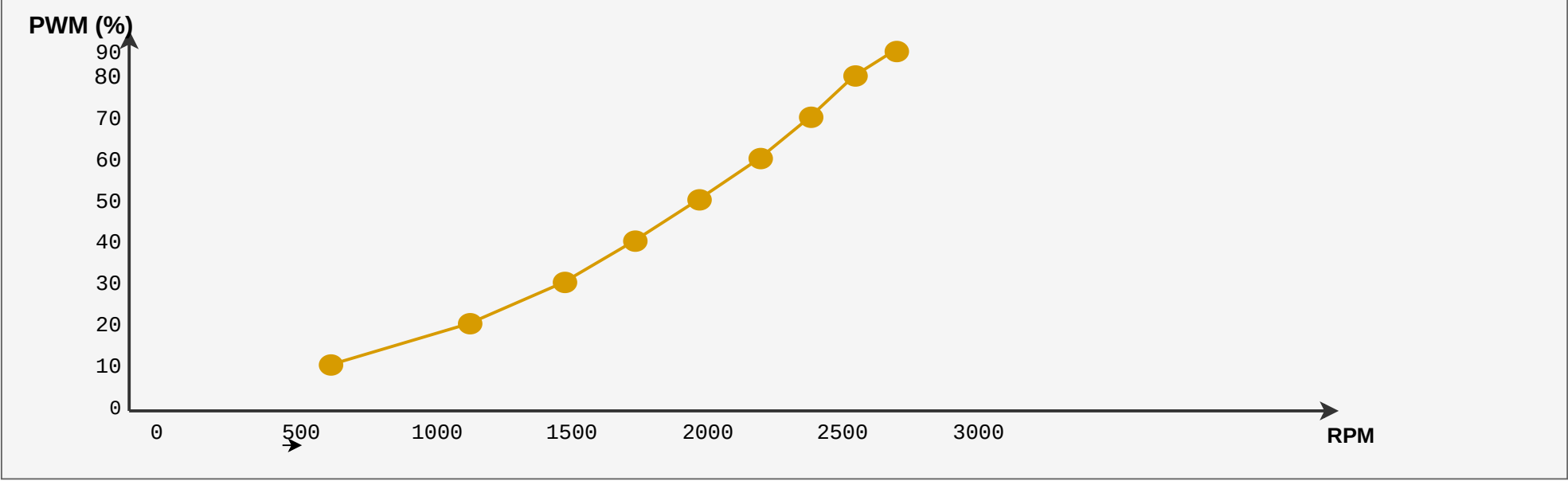
THE PROBLEM



Feedforward Lookup Table

CALIBRATION DATA (Open-Loop Measurement) GRAPHICAL REPRESENTATION

Pt	RPM	PWM (%)
1	740	10
2	1250	20
3	1600	30
4	1860	40
5	2100	50
6	2325	60
7	2515	70
8	2675	80
9	2830	90



IMPLEMENTATION

```
const float ff_rpm_points[] = {740, 1250, 1600, 1860, 2100, 2325, 2515, 2675, 2830};
const float ff_pwm_points[] = {10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0, 80.0, 90.0};

float calculate_feedforward_pwm(float rpm) {
    if (rpm <= ff_rpm_points[0]) return ff_pwm_points[0];
    if (rpm >= ff_rpm_points[8]) return ff_pwm_points[8];

    for (int i = 0; i < 8; i++) {
        if (rpm >= ff_rpm_points[i] && rpm <= ff_rpm_points[i+1]) {
            float x0 = ff_rpm_points[i];
            float y0 = ff_pwm_points[i];
            float x1 = ff_rpm_points[i+1];
            float y1 = ff_pwm_points[i+1];
            return y0 + (rpm - x0) * (y1 - y0) / (x1 - x0);
        }
    }
    return 0.0f;
}
```

INTEGRATION WITH PID

```
float ff_term = calculate_feedforward_pwm(setpoint_rpm); // ~80% of output
float p_term = Kp * error;
float i_term = Ki * integral;
float d_term = Kd * (-delta_measured / dt);

float output = ff_term + p_term + i_term + d_term; // Combine
```

ANTI-WINDUP WITH FEEDFORWARD

```
integral_headroom = PID_OUTPUT_MAX - ff_term; // e.g., 95 - 85 = 10%
integral_floor = PID_OUTPUT_MIN - ff_term; // e.g., 10 - 85 = -75%

max_integral = integral_headroom / Ki;
min_integral = integral_floor / Ki;

if (integral > max_integral) integral = max_integral;
if (integral < min_integral) integral = min_integral;
```

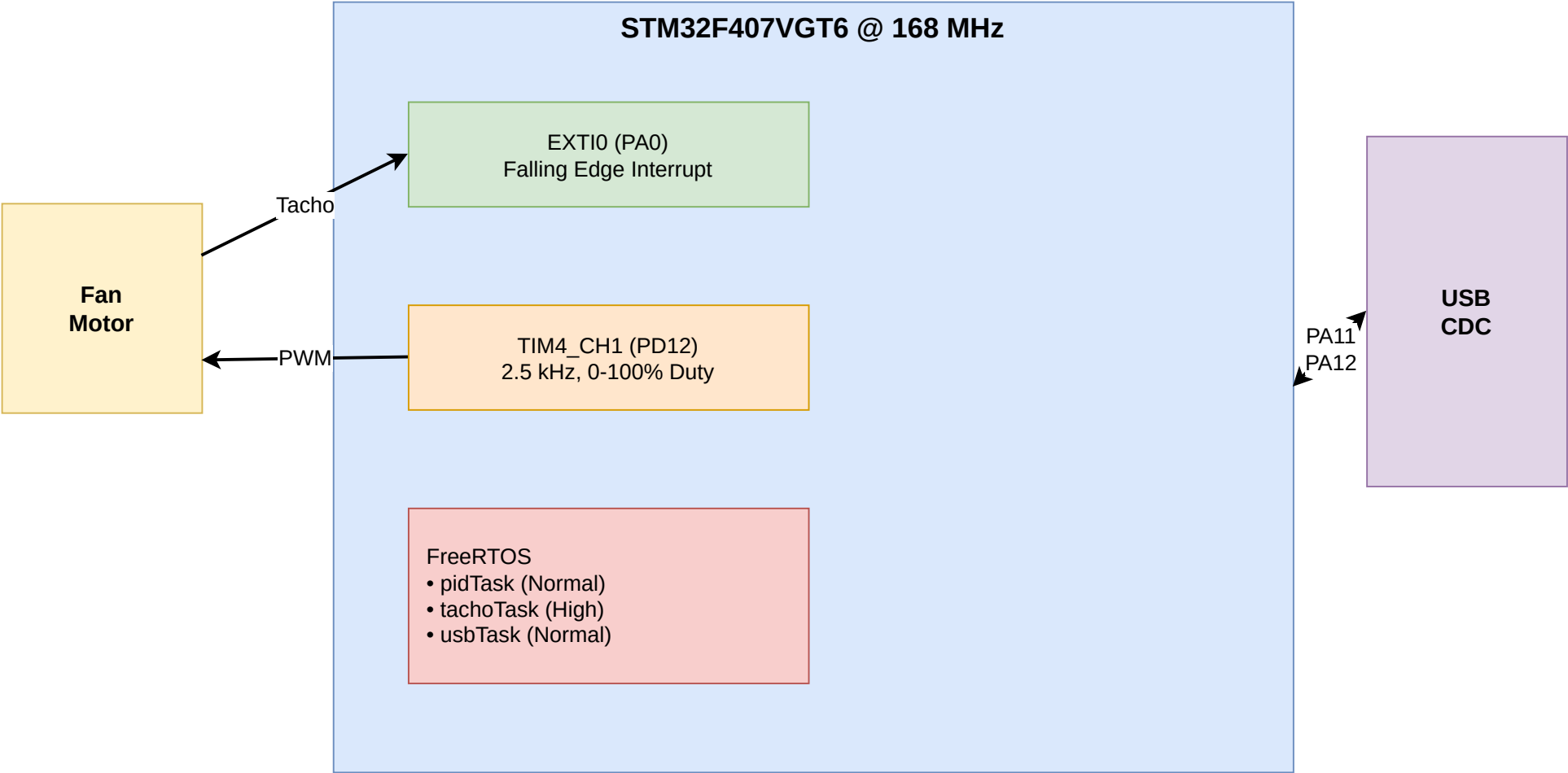
EXAMPLE: RPM = 1800

Between Point 3 (1600, 30%) and Point 4 (1860, 40%)

PWM = 30.0 + (1800 - 1600) × (40.0 - 30.0) / (1860 - 1600)
= 30.0 + 200 × 10.0 / 260
= 37.69%

Result: 37.69% PWM

STM32 Fan Control System Architecture



Peripheral Configuration

TIM2 - Microsecond Timer (Tachometer Timing)

Clock Source: APB1 Timer (84 MHz)
Prescaler: 83
Counter Freq: $84\text{MHz} / (83+1) = 1\text{ MHz} \rightarrow 1\mu\text{s}$ resolution
Mode: 32-bit free-running
ARR: 0xFFFFFFFF (max)

TIM4 - PWM Generation (Fan Control)

Clock Source: APB1 Timer (84 MHz)
Prescaler: 83
Timer Freq: $84\text{MHz} / (83+1) = 1\text{ MHz}$
ARR (Period): 399
PWM Freq: $1\text{MHz} / (399+1) = 2.5\text{ kHz}$
Channel: CH1 (PD12)
Mode: PWM Mode 1
Duty Cycle: $\text{CCR1} / 400 \times 100\%$

USB OTG FS - CDC Virtual COM Port

Pins: PA11 (DM), PA12 (DP)
Clock: 48 MHz (PLL /Q=7)
Class: CDC (Virtual COM Port)
Buffer: TX: 2048 bytes, RX: 2048 bytes
Commands: s, m, p/i/d, save

GPIO - Tachometer Input

Pin: PA0
Mode: External Interrupt (EXTI0)
Trigger: Falling Edge
Pull: Pull-up
Priority: 5

Configuration Summary:

Timers:

- TIM2: Microsecond timestamp counter for tachometer pulse timing
 - 32-bit counter prevents overflow for ~71 minutes
 - 1 μ s resolution sufficient for RPM measurement (740-2830 RPM)
- TIM4: PWM generation for fan speed control
 - 2.5 kHz frequency
 - 400 steps (0-399) provides 0.25% duty cycle resolution

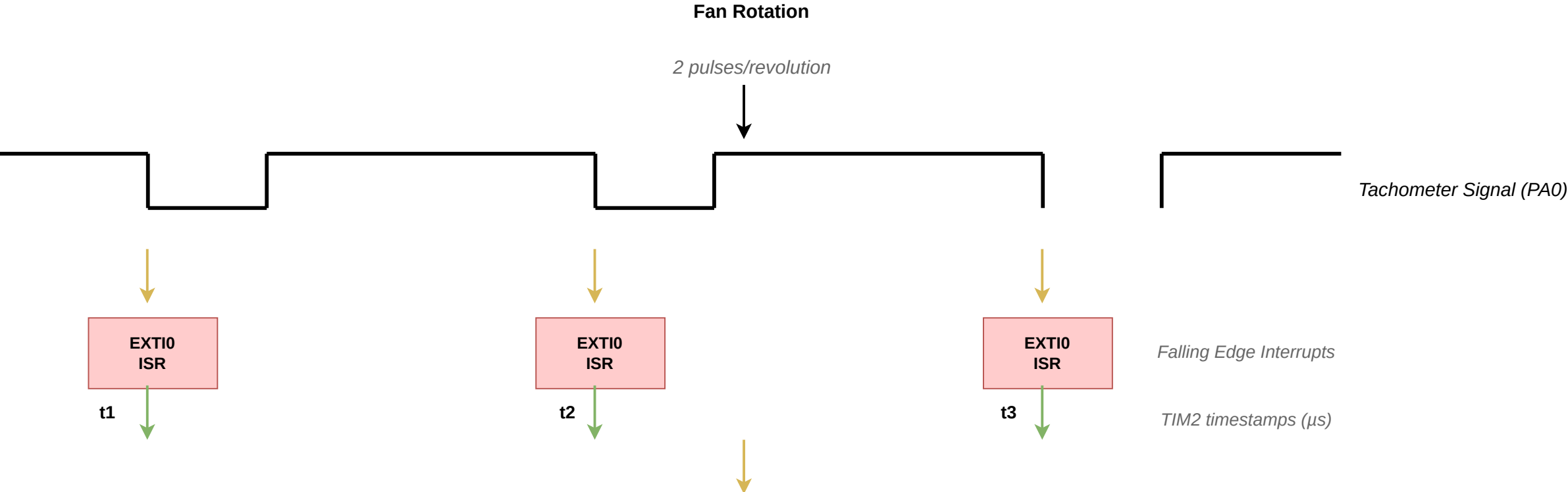
Communication:

- USB CDC: Virtual COM port for real-time telemetry and control
 - No UART-to-USB adapter required
 - 2KB buffers handle burst telemetry at 4 Hz

Interrupt:

- EXTI0: Falling edge detection on tachometer signal
 - Priority 5 ensures timely response
 - Pull-up resistor keeps signal high when fan is stopped

Tachometer to RPM Conversion



Example Calculation:

For 1500 RPM fan:

- Δt between pulses = 20,000 μs
- $RPM = 60,000,000 / (20,000 \times 2)$
- $RPM = 60,000,000 / 40,000$
- $RPM = 1500$

Notes:

- TIM2 runs at 84 MHz (APB1 timer)
- Prescaler = 83 $\rightarrow$ 1 μs resolution
- 2 pulses per revolution (fan spec)
- First pulse ignored (no Δt yet)
- 1000ms timeout $\rightarrow$ fan stopped

tachoTask Processing

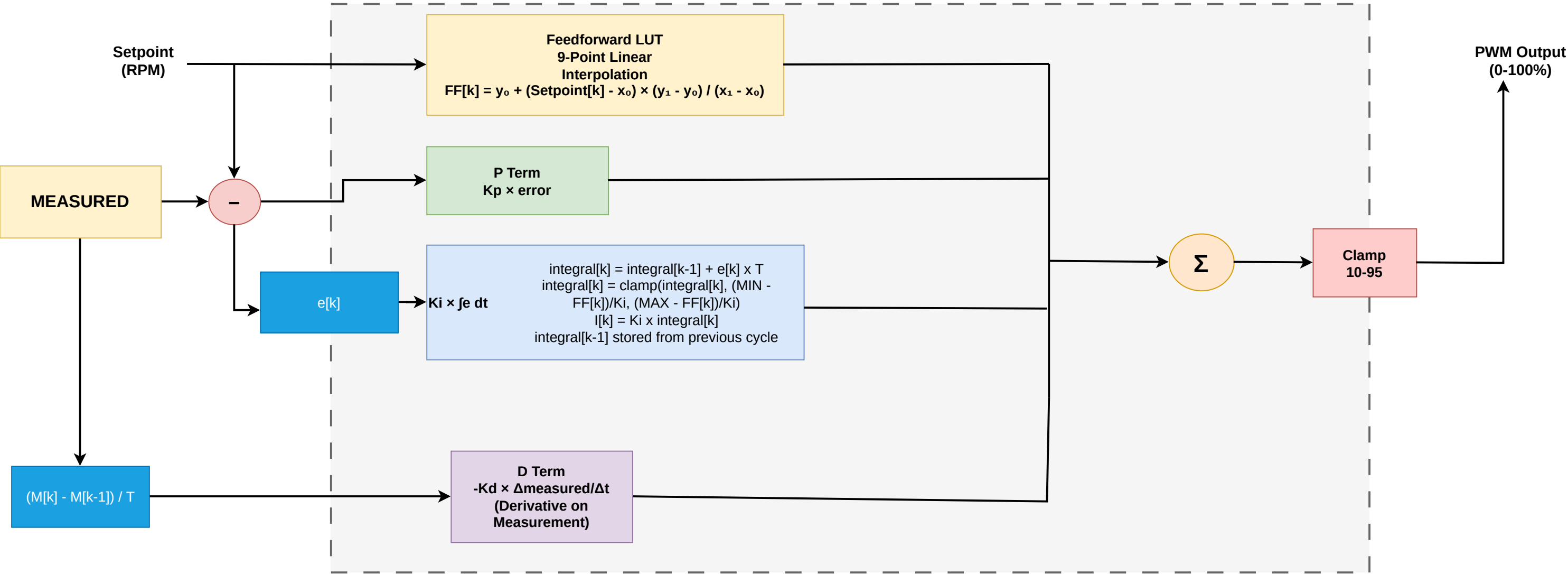
$$\Delta t = t2 - t1$$

$$RPM = \frac{60,000,000}{\Delta t \times 2}$$

Timeout (1000ms)?
└─> Yes: RPM = 0
└─> No: Update measured_rpm

First pulse after startup?
└─> Ignore (rejected)

PID Control Architecture (40 Hz Loop)



Notes:

- Sample Time: 25 ms (40 Hz)
- Anti-windup: Integral limited by $(\text{MAX} - FF_term) / K_i$
- Derivative on measurement prevents setpoint kick
- FF LUT: 740-2830 RPM $\rightarrow$ 10-90% PWM

Task Priorities & Synchronization

